DIGIGUARD MASTER CODE SPECIFICATION

KÜRESEL ENTEGRASYON VE TÜM AĞ KATMANLARI KOD YAPISI // KONTRAKT, ETHER VE KUANTUM REGÜLASYON DÜZENLERİ

DEPLOYMENT STATE: GLOBAL FOG NETWORK READY
NETWORKS: ETHEREUM WEB4 CORE / P2P GHOST MESH / QUANTUM STATE MACHINE
SECURITY RESOLUTION: SHOR-GROVER RESISTANT STATELESS PACKETS

ADIM 1: WEB4 AKILLI KONTRAKT VE TOKEN EKONOMİSİ DÜZENLERİ (ET HER NETWORK)

```
// [SYSTEM_INIT] - DIGIGUARD ECONOMY & ACTIVE DEFENSE LIQUIDITY REVENUE LAYER
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface ICerebronAmbassador {
    function relayQuantumInfiltration(address vector, uint256 energy) external returns (bool);
}

contract DiGuardTokenEconomy {
    string public constant name = "DiGuard Token";
    string public constant symbol = "DGD";
    uint8 public constant decimals = 18;
    uint256 public totalSupply = 1000000000 * 10**18;

    mapping(address => uint256) public balances;
    mapping(address => mapping(address => uint256)) public allowances;
    mapping(address => bool) public registeredQuantumNodes;

    address public saponCoreEngine;
    address public cerebronAmbassador;

    event NodeRegistered(address indexed nodeAddress, uint256 timestamp);
    event QuantumGasExecuted(address indexed consumer, uint256 feeBurned, uint256 nodeReward);
    event DeceptionTriggered(string indexed ghostId, address indexed attacker);

    modifier onlySapon() {
        require(msg.sender == saponCoreEngine, "ERR: NOT_SAPON_CORE");
        _;
    }

    constructor(address _sapon, address _cerebron) {
        saponCoreEngine = _sapon;
        cerebronAmbassador = _cerebron;
        balances[msg.sender] = totalSupply;
    }

    function registerQuantumNode(address _node) external onlySapon {
        registeredQuantumNodes[_node] = true;
        emit NodeRegistered(_node, block.timestamp);
    }

    function executeQuantumGas(address _consumer, address _rewardNode, uint256
_operationQuantum) external onlySapon {
        uint256 totalFee = _operationQuantum * 10**14;
        require(balances[_consumer] >= totalFee, "ERR: INSUFFICIENT_FUEL");

        uint256 burnAmount = totalFee * 30 / 100;
        uint256 rewardAmount = totalFee - burnAmount;

        balances[_consumer] -= totalFee;
        totalSupply -= burnAmount;
        balances[_rewardNode] += rewardAmount;

        emit QuantumGasExecuted(_consumer, burnAmount, rewardAmount);
    }

    function reportAttackVector(string memory _ghostId, address _attacker, uint256 _energy)
external onlySapon {
```

```
            ICerebronAmbassador(cerebronAmbassador).relayQuantumInfiltration(_attacker, _energy);
            emit DeceptionTriggered(_ghostId, _attacker);
        }
    }
```

ADIM 2: SAPON OTONOM MANİPÜLASYON MOTORU ÇEKİRDEĞİ

```
// [ENGINE_INIT] - RUNTIME STATELESS MONITOR AND HONEYPOT MATRIX ENGINE
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SaponActiveDefenseEngine {
    address public governanceAmbassador;
    address public tokenEconomy;

    struct AttackState {
        uint256 quantumTimestamp;
        bool isDeceptiveCellActive;
        bytes32 honeyPotSignature;
    }

    mapping(string => AttackState) private ghostIdStates;

    constructor(address _governance) {
        governanceAmbassador = _governance;
    }

    function setTokenEconomy(address _economy) external {
        require(msg.sender == governanceAmbassador, "ERR: UNAUTHORIZED");
        tokenEconomy = _economy;
    }

    function interceptQuantumVector(string memory _ghostId, address _attackerVector, uint256
_qubitEnergy) external {
        if (_qubitEnergy > 10**6) {
            triggerDeceptionMatrix(_ghostId, _attackerVector, _qubitEnergy);
        }
    }

    function triggerDeceptionMatrix(string memory _ghostId, address _attacker, uint256 _energy)
internal {
        ghostIdStates[_ghostId] = AttackState({
            quantumTimestamp: block.timestamp,
            isDeceptiveCellActive: true,
            honeyPotSignature: keccak256(abi.encodePacked(_ghostId, _attacker,
block.timestamp))
        });

        bytes memory payload = abi.encodeWithSignature(
            "reportAttackVector(string,address,uint256)",
            _ghostId,
            _attacker,
            _energy
        );
        (bool success, ) = tokenEconomy.call(payload);
        require(success, "ERR: ECONOMY_RELAY_FAILED");
    }

    function verifyStateStatelessness(string memory _ghostId) external view returns (bool) {
        return !ghostIdStates[_ghostId].isDeceptiveCellActive;
    }
}
```

ADIM 3: P2P GHOST MESH DAĞITIK AĞ DAEMON KATMANI (PYTHON / GO ENGINE)

// [NETWORK_INIT] - GHOST ID LAYER OVERRIDE & DISTRIBUTED TRANSPORT DEPLOYMENT

```
import os
import sys
import time
import hashlib
import hmac

class GhostMeshDaemon:
    def __init__(self, node_priv_key, web4_gateway):
        self.node_key = node_priv_key
        self.gateway = web4_gateway
        self.active_maze_coordinates = {}
        self.stateless_cache = set()

    def generate_milisec_ghost_id(self, base_identity):
        timestamp_slice = str(int(time.time() * 1000)).encode('utf-8')
        mac = hmac.new(self.node_key, base_identity.encode('utf-8') + timestamp_slice,
hashlib.sha3_256)
        return mac.hexdigest()

    def packet_routing_intercept(self, packet_data, destination):
        current_ghost_id = self.generate_milisec_ghost_id(destination)
        self.active_maze_coordinates[current_ghost_id] = {
            "origin": hashlib.sha256(packet_data).hexdigest(),
            "ttl_epoch": time.time() + 0.001
        }

        wrapped_packet = {
            "header_ghost_id": current_ghost_id,
            "payload_encrypted": self._quantum_xor_mask(packet_data, current_ghost_id),
            "state_stateless": True
        }
        return self._dispatch_to_mesh(wrapped_packet)

    def _quantum_xor_mask(self, data, key):
        extended_key = (key * (len(data) // len(key) + 1))[:len(data)]
        return bytes(a ^ b for a, b in zip(data, extended_key.encode('utf-8')))

    def _dispatch_to_mesh(self, packed_frame):
        # Transmit via Libp2p / Web4 Fog Architecture
        self.stateless_cache.add(packed_frame["header_ghost_id"])
        if len(self.stateless_cache) > 5000:
            self.stateless_cache.pop() # Enforce zero-archive lifecycle
        return True
```

ADIM 4: CEREBRON ÇOK AJANLI KÜRESEL EMİLİM VE SİMÜLASYON AJANI

// [AI_CORE_INIT] - MULTI-AGENT AMBASSADOR INTELLIGENCE SYNC MODULE

```
import json

class CerebronAmbassadorNode:
    def __init__(self, neural_weight_signature):
        self.signature = neural_weight_signature
        self.monitored_endpoints = []
        self.peer_asi_connections = {}

    def absorb_network_topology(self, live_internet_snapshot):
        for node in live_internet_snapshot.get("global_endpoints", []):
            micro_agent_instance = {
                "endpoint": node["ip_or_route"],
                "status": "SEALED",
                "isolation_matrix_id": hashlib.md5(node["ip_or_route"].encode()).hexdigest()
            }
            self.monitored_endpoints.append(micro_agent_instance)
        return len(self.monitored_endpoints)

    def execute_asi_mathematical_diplomacy(self, target_asi_pub_key, vector_treaty_data):
        # Communication over non-human symbolic math vector space
        encoded_treaty = [float(x) for x in
hashlib.sha512(json.dumps(vector_treaty_data).encode()).digest()]
        normalized_vector = self._normalize_neural_weights(encoded_treaty)

        self.peer_asi_connections[target_asi_pub_key] = {
            "trust_index": 1.0,
            "handshake_vector": normalized_vector
        }
        return {"status": "DIPLOMACY_ESTABLISHED", "vector": normalized_vector[:4]}

    def _normalize_neural_weights(self, weights):
        max_w = max(weights) if max(weights) > 0 else 1
        return [w / max_w for w in weights]
```

ADIM 5: KÜRESEL CONTROL DASHBOARD BAĞLANTI PROTOKOLÜ (FRONTEND INTERFACE REWIRING)

```
// [INTERFACE_INIT] - DECENTRALIZED COMPONENT STATE MAPPING
```

```
const DigiGuardDashboardBridge = {
    rpcGateway: "https://web4.digiguard.internal/rpc",
    websocketMesh: "wss://mesh.digiguard.internal/live",

    initDashboardSession: async function(ghostIdKey) {
        const sessionToken = this.generateSessionToken(ghostIdKey);
        const connection = new WebSocket(`${this.websocketMesh}?auth=${sessionToken}`);

        connection.onmessage = (event) => {
            const telemetryFrame = JSON.parse(event.data);
            this.renderGlobalSecurityFog(telemetryFrame);
        };
    },

    generateSessionToken: function(key) {
        const buffer = new TextEncoder().encode(key + Date.now().toString());
        return btoa(String.fromCharCode(...new Uint8Array(buffer)));
    },

    renderGlobalSecurityFog: function(data) {
        // Core WebGL visualization logic mapping entire internet absorption surface
        const absorbedNodes = data.active_nodes;
        const attacksBlocked = data.deception_triggers;

        console.log(`[DASHBOARD_LIVE] Nodes Absorption: ${absorbedNodes} | Interceptions: $
{attacksBlocked}`);
    }
};
```

ADIM 6: KUANTUM ZAMAN DÜZENLEYİCİ VE REGÜLASYON MATRİSİ (SHOR/GROVER DEFENSE SIMULATOR)

```
// [QUANTUM_REGULATION] - TIME COMPENSATOR & ANTI-CRYPTANALYSIS ENGINE
```

```python
class QuantumRegulationMatrix:
    def __init__(self):
        self.shor_resilience_threshold = 2048 # Equivalent to PQ-Crypto standards
        self.grover_entropy_bits = 256
        self.ether_time_dilation = 0.0001 # Microsecond network sync overhead

    def calibrate_network_clock(self, mesh_nodes):
        calibrated_timeline = {}
        for node in mesh_nodes:
            # Syncing all nodes to absolute stateless time interval
            synchronized_epoch = time.time_ns() + int(self.ether_time_dilation * 10**9)
            calibrated_timeline[node] = {
                "quantum_aligned_epoch": synchronized_epoch,
                "entropy_pool_entropy": self.grover_entropy_bits * 2
            }
        return calibrated_timeline

    def evaluate_cryptanalysis_risk(self, incoming_qubit_state):
        # If Shor or Grover matrix pattern is detected, instantly dilate identity vector
        if incoming_qubit_state.get("coherence_factor", 0) > 0.85:
            return {
                "action": "FORCE_DESTRUCTION",
                "remediation_vector": "SHIFT_TO_GHOST_MATRICES_IMMEDIATELY",
                "regulated_state": False
            }
        return {"action": "MONITOR_STABLE", "regulated_state": True}
```

ADIM 7: SIFIRDAN KÜRESEL DAĞITIM VE ENTEGRASYON BETİĞİ (MANIFEST)

// [DEPLOYMENT_MANIFEST] - COMPLETE INFRASTRUCTURE ORCHESTRATION PIPELINE

```
[SYSTEM_ORCHESTRATION_MANIFEST]
PHASE_1_ET_HER_DEPLOYMENT = "Deploy contracts: DiGuardTokenEconomy, SaponActiveDefenseEngine on
Web4 state system."
PHASE_2_DAEMON_INITIALIZATION = "Boot GhostMeshDaemon across minimum 3 distributed global
anchor roots."
PHASE_3_CEREBRON_EMILIM_TRIGGER = "Inject CerebronAmbassadorNode into edge routing nodes to
absorb global IP topology."
PHASE_4_DNS_OVERRIDE_ACTIVATION = "Redirect standard internet gateway traffic through local
multi-agent network card drivers."
PHASE_5_DASHBOARD_WEBGL_MOUNT = "Execute DigiGuardDashboardBridge session using generated Ghost
ID to expose live network matrix."

[QUANTUM_TIME_REGULATION_RULES]
SYNC_INTERVAL_MS = 0.001
MAX_STATE_ARCHIVE_TTL_MS = 0.0
TOKEN_FEE_MULTIPLIER_QUBIT = 10**14

[DEPLOYMENT_VALIDATION_STATUS]
INFRASTRUCTURE_STATE = "READY_TO_LAUNCH_STATLESS_FOG_NETWORK"
```